Call Me Maybe: Constrained Decoding e Function Calling su Small Language Models

Analisi del Subject (v1.5), Fondamenti Teorici dalla Letteratura Scientifica (arXiv) e Architettura Implementativa per Qwen3-0.6B

---

1. Analisi Approfondita del Subject "Call Me Maybe" (v1.5)

Il progetto **Call Me Maybe** affronta uno dei problemi centrali dell'ingegneria dei Large Language Models (LLM) moderni: trasformare un modello autoregressivo generativo, progettato per produrre testo libero probabilistico, in un traduttore deterministico capace di convertire richieste in linguaggio naturale in **chiamate a funzione strutturate (Function Calling)** conformi a uno schema JSON rigoroso.

L'aspetto sfidante del progetto risiede nelle dimensioni ridotte del modello di riferimento: `Qwen/Qwen3-0.6B` (~596 milioni di parametri totali). Come evidenziato nel capitolo III.3 del subject, un modello di taglia sub-miliardo interrogato tramite semplice *prompting* produce JSON sintatticamente valido e fedele allo schema solo nel ~30% dei casi. Per raggiungere il requisito di produzione del **100% di validità JSON** e oltre il **90% di accuratezza semantica**, il progetto impone l'implementazione da zero di un motore di **Constrained Decoding** (decodifica vincolata) che manipola direttamente i logit del modello passo dopo passo.

1.1 Requisiti Architetturali, Vincoli e Regole di Validazione

L'analisi sistematica dei Capitoli IV, V, VI e VII del subject evidenzia vincoli stringenti che determinano l'architettura del software:

Categoria	Specifiche Obbligatorie del Subject	Implicazioni Ingegneristiche
Ambiente e Linguaggio	Python ≥ 3.10, gestione pacchetti tramite `uv` (la valutazione esegue esclusivamente `uv sync`).	Configurazione rigorosa di `pyproject.toml` e `uv.lock`; isolamento completo in virtualenv.
Librerie Consentite vs Vietate	Consentite: `pydantic` (obbligatoria per tutte le classi), `numpy`, `json`, `llm_sdk`. **Vietate**: `outlines`, `dspy`, `transformers`, `torch`, `huggingface`.	L'algoritmo di mascheramento dei logit, il parser di stato JSON e la selezione dei token devono essere scritti interamente da zero usando solo Python nativo e `numpy`.
Interfaccia `llm_sdk`	Classe `Small_LLM_Model`: metodi pubblici `get_logits_from_input_ids`, `get_path_to_vocab_file`, `encode`, `decode`. Vietato l'uso di attributi/metodi privati (`_...`).	Il modello è trattato come una *black-box* che mappa una lista di `int` (token ID) in una lista di `float` (logit sul vocabolario).

Qualità del Codice e Linting	`flake8` + `mypy` con flag: `--warn-return-any --warn-unused-ignores --ignore-missing-imports --disallow-untyped-defs --check-untyped-defs` (consigliato `--strict`). Docstring PEP 257.	Ogni funzione e metodo deve avere annotazioni di tipo esplicite; nessun ritorno implicito di `Any` da chiamate JSON o NumPy non tipizzate.
Divieto di Euristiche ("Medieval Magic")	La scelta della funzione da invocare e l'estrazione degli argomenti devono avvenire tramite i logit dell'LLM, mai tramite regex o keyword matching sul prompt.	Anche la selezione del nome della funzione deve avvenire calcolando le probabilità dell'LLM sui token che compongono i nomi validi definiti nello schema.
Performance e Affidabilità	100% JSON valido e conforme allo schema; ≥ 90% di accuratezza nella selezione della funzione e degli argomenti; esecuzione completa in **meno di 5 minuti**; zero crash su input corrotti.	Poiché ogni chiamata a `get_logits_from_input_ids` esegue una forward pass costosa, è essenziale minimizzare il numero di invocazioni tramite *deterministic token fast-forwarding*.

1.2 Formato dei Dati di Input e Output

Il programma viene invocato tramite CLI (`uv run python -m src [--functions_definition ...] [--input ...] [--output ...]`) e legge due file di input principali:

- `functions_definition.json`: Un array di descrittori di funzione. Ogni elemento definisce `name` (stringa), `description` (stringa), `parameters` (mappa chiave-oggetto dove ogni parametro specifica il proprio `{"type": "number" | "string" | "boolean" | ...}`) e `returns`.
- `function_calling_tests.json`: Un array di oggetti contenenti la chiave `"prompt"` con la richiesta in linguaggio naturale (es. `"What is the sum of 2 and 3?"`).

L'output generato deve essere un singolo file JSON contenente un array di oggetti aventi **esattamente tre chiavi**: `prompt` (la stringa originale), `name` (il nome della funzione scelta) e `parameters` (l'oggetto JSON contenente tutti gli argomenti richiesti con i tipi esatti previsti dallo schema).

2. Chiarimento Teorico: Constrained Decoding vs. Jump-Forward Decoding

Il subject ufficiale (Capitolo V.3.3) prescrive l'uso del **Constrained Decoding**. In sede di valutazione (peer defense) a 42, padroneggiare la differenza esatta tra Constrained Decoding e Jump-Forward Decoding — e comprendere come la loro combinazione permetta di accelerare drasticamente l'esecuzione in `call me maybe` — rappresenta un punto di forza decisivo.

> **Distinzione Concettuale Chiave per la Peer Evaluation**
>
> Il **Constrained Decoding** (**Willard & Louf, 2023**; **Geng et al., 2023**) è una tecnica di *controllo strutturale* su un singolo modello tramite mascheramento dei logit a $-\infty$. Quando viene applicato a uno schema JSON rigido, abilita il cosiddetto **Jump-Forward (o Fast-Forward) Decoding** (**Zheng et al., 2023**): un meccanismo in cui l'automa grammaticale riconosce con certezza matematica del 100% i token strutturali obbligati, iniettandoli nel contesto senza interrogare la rete neurale.

2.1 Grammar-Constrained Decoding (Willard & Louf, 2023; Geng et al., 2023)

Formalizzato nella sua forma moderna efficiente da **Willard & Louf (2023)** (autori della libreria *Outlines*, citata e vietata dal subject proprio perché risolverebbe l'intero progetto in tre righe) e da **Geng et al. (2023)**, il **Constrained Decoding** definisce un linguaggio formale $L \subseteq \Sigma^*$ (ad esempio l'insieme di tutte le stringhe JSON conformi a uno schema dato). Dato un prefisso generato $x \in \Sigma^*$ e il vocabolario del modello $V \subset \Sigma^+$, l'insieme dei token validi $V_{valid}(x)$ al passo corrente è definito come l'insieme dei token $v \in V$ che costituiscono una continuazione valida verso almeno una stringa completa in L:

$$V_{valid}(x) = \{ v \in V \mid \exists y \in \Sigma^* \text{ tale che } (x \cdot v \cdot y) \in L \}$$

Prima di selezionare il prossimo token, il decodificatore applica una maschera booleana al vettore dei logit $z \in \mathbb{R}^{|V|}$ prodotti dall'LLM, ponendo $z_v = -\infty$ per ogni $v \notin V_{valid}(x)$. In questo modo la probabilità post-Softmax di qualunque token sintatticamente o strutturalmente invalido diventa esattamente zero.

2.2 Jump-Forward Decoding tramite Compressed FSM (Zheng et al., 2023 - SGLang)

Qui la decodifica vincolata incontra l'ottimizzazione di velocità. Come dimostrato da **Zheng et al. (2023)** nel paper di **SGLang**, durante la generazione di un oggetto JSON vincolato da uno schema, gran parte dei caratteri è **totalmente deterministica** (ha un solo cammino possibile nell'automa a stati finiti).

Consideriamo un esempio concreto in `call me maybe`. Supponiamo che l'LLM abbia appena scelto la funzione `fn_add_numbers`, i cui parametri nello schema sono `a` e `b`. L'oggetto di output (o la chiamata generata dal modello) deve obbligatoriamente proseguire con la stringa:

```
{"name": "fn_add_numbers", "parameters": {"a":
```

Chiedere all'LLM di generare uno alla volta i token per `",`, `"parameters"`, `:`, `{"a":` invocando 6 o 7 volte `get_logits_from_input_ids()` solo per mascherare il 99,999% del vocabolario e forzare l'unico token valido è uno spreco computazionale enorme! Con il **Jump-Forward Decoding**:

- Quando lo stato dello schema impone una sequenza di caratteri fissa (es. l'apertura del JSON, i nomi delle chiavi dei parametri `"a":`, le virgole separatore `, "b":`), **non invochiamo affatto l'LLM**.
- Codifichiamo direttamente i token della stringa deterministica nel buffer `input_ids` (avanzando lo stato dell'automa).
- Invochiamo `get_logits_from_input_ids()` **esclusivamente nei punti di ramificazione semantica** (*branching points*): la scelta del nome della funzione e la generazione dei valori dei singoli parametri (es. `40` e `2`).

Questo approccio riduce il numero di forward pass dell'LLM del **70%–85%**, garantendo il completamento dell'intera suite di test in pochi secondi (ben al di sotto del limite di 5 minuti del subject).

Caratteristica	Constrained Decoding (Willard & Louf, 2023)	Jump-Forward Constrained (Zheng et al., 2023)
Obiettivo Primario	Garantire il 100% di conformità a una grammatica/schema tramite logit masking.	Unire conformità al 100% e massima velocità saltando le forward pass sui tratti fissi dello schema.
Modelli Coinvolti	1 modello + Automa a Stati Finiti (FSM) / Parser.	1 modello + Compressed FSM.
Applicabilità in Call Me Maybe	**Obbligatorio** per scegliere la funzione ed estrarre i valori tipizzati.	**Altamente raccomandato** per rispettare il requisito < 5 minuti su CPU.

3. Architettura Transformer Autoregressiva, Logits e Qwen3-0.6B

Per progettare correttamente il sistema di decodifica, occorre comprendere come `Qwen/Qwen3-0.6B` trasforma una sequenza di interi in una distribuzione di probabilità sul prossimo token (**Vaswani et al., 2017**; **Yang et al., 2025**).

3.1 Dai Token ID ai Logits

Dato un prompt convertito in una sequenza di t token ID $x_{1:t} = (x_1, x_2, \ldots, x_t)$ dove ogni $x_i \in \{0, 1, \ldots, |V| - 1\}$, il modello Transformer applica un layer di embedding seguito da 28 blocchi di *Grouped-Query Attention* (GQA) e reti Feed-Forward (SwiGLU). L'ultimo stato nascosto $h_t \in \mathbb{R}^{1024}$ viene moltiplicato per la matrice di proiezione $W_{LM} \in \mathbb{R}^{|V| \times 1024}$, producendo il vettore dei **logits**:

$$z = W_{LM} \cdot h_t \in \mathbb{R}^{|V|}$$

Questo vettore z è esattamente ciò che restituisce la chiamata `model.get_logits_from_input_ids(input_ids)`. I logit sono punteggi grezzi non normalizzati (possono assumere qualsiasi valore reale positivo o negativo, ad es. da −25.4 a +32.1). In una pipeline standard, i logit vengono convertiti in probabilità tramite la funzione **Softmax** scalata da una temperatura $T > 0$:

$$P(x_{t+1} = i \mid x_{1:t}) = exp(z_i / T) / \sum_{j=1..|V|} exp(z_j / T)$$

3.2 Greedy Decoding vs. Sampling nel Function Calling

Nel contesto del Function Calling deterministico (dove cerchiamo la risposta più probabile e riproducibile per i test della moulinette), non è necessario né desiderabile campionare casualmente (*stochastic sampling*), poiché il campionamento casuale può selezionare cifre errate o caratteri spuri a bassa probabilità. Si utilizza invece il **Greedy Constrained Decoding** (equivalente al limite $T \to 0^+$):

$$x_{t+1} = arg\ max_{i \in V_{valid}(x_{1:t})} z_i$$

Poiché la funzione esponenziale è strettamente monotona crescente, il token che massimizza la probabilità Softmax tra i token validi è esattamente il token che massimizza il logit grezzo z_i all'interno del sottoinsieme

V_{valid}! Ciò significa che non serve nemmeno calcolare `exp()` su tutto il vocabolario: basta applicare la maschera dei token validi a z e calcolare `np.argmax(masked_logits)`.

4. Tokenizzazione Byte-Level BPE e il Problema dei Confini (Boundary Misalignment)

Il secondo pilastro tecnico del progetto — nonché requisito esplicito della **Bonus Part** (Capitolo VII: *"Recoding the tokenizer: avoiding direct use of encode and decode in the main code, instead using get_logits_from_input_ids and get_path_to_vocab_file"*) — riguarda la comprensione profonda del tokenizer di Qwen3 (**Sennrich et al., 2016**; **Radford et al., 2019**).

4.1 Come funziona il Vocabolario Byte-Level BPE di Qwen3

Il file restituito da `get_path_to_vocab_file()` è un dizionario JSON che mappa la rappresentazione testuale di ciascun token al suo ID numerico intero (per Qwen3 sono circa 151.643 token base più i token speciali, fino a ~151.936). A differenza dei tokenizer ingenui, Qwen utilizza il **Byte-Level Byte-Pair Encoding (BBPE)** introdotto con GPT-2 (**Radford et al., 2019**):

- Ogni stringa UTF-8 viene prima vista come una sequenza di byte (0–255).
- Per evitare caratteri di controllo invisibili o spazi bianchi ambigui nelle chiavi di `vocab.json`, ognuno dei 256 byte viene mappato biunivocamente a un carattere Unicode stampabile.
- In particolare:
  - Il byte `0x20` (spazio ASCII `' '`) è rappresentato dal carattere Unicode **Ġ** (`Ġ`, codice 288). Quindi il token `"Ġsum"` corrisponde alla stringa reale `" sum"` (con spazio iniziale).
  - Il byte `0x0A` (newline `'\n'`) è rappresentato da **Ċ** (`Ċ`, codice 266).
  - Il byte `0x09` (tab `'\t'`) è rappresentato da **ĉ** (`ĉ`, codice 265).

4.2 Implementazione Pratica del Decoder/Encoder da `vocab.json` (Anche per il Bonus)

Per sapere quale stringa reale viene aggiunta al nostro JSON quando l'LLM sceglie il token `token_id`, dobbiamo invertire la mappa `byte_to_unicode` standard di GPT-2/Qwen. Di seguito è riportato il codice Python completo, tipizzato per `mypy --strict`, che ricostruisce il mapping esatto dei byte e implementa sia la conversione del vocabolario sia un encoder/decoder autonomo:

```
from typing import Dict, List
import json


def build_byte_decoder() -> Dict[str, int]:
    """Costruisce la mappa inversa da caratteri Unicode BPE (es. 'Ġ') a byte UTF-8 (0-
255)."""
    bs: List[int] = (
        list(range(ord("!"), ord("~") + 1))
        + list(range(ord("¡"), ord("¬") + 1))
        + list(range(ord("®"), ord("ÿ") + 1))
    )
    cs: List[int] = bs[:]
```

```
    n = 0
    for b in range(256):
        if b not in bs:
            bs.append(b)
            cs.append(256 + n)
            n += 1
    return {chr(c): b for b, c in zip(bs, cs)}


class RawVocabTokenizer:
    """Gestisce il vocabolario BPE direttamente da vocab.json senza usare encode/decode
dell'SDK."""

    def __init__(self, vocab_path: str) -> None:
        with open(vocab_path, "r", encoding="utf-8") as f:
            raw_vocab: Dict[str, int] = json.load(f)

        self.byte_decoder: Dict[str, int] = build_byte_decoder()
        self.byte_encoder: Dict[int, str] = {v: k for k, v in self.byte_decoder.items()}
        self.token_to_id: Dict[str, int] = raw_vocab
        self.id_to_str: Dict[int, str] = {}

        for bpe_token, token_id in raw_vocab.items():
            # Converte ogni token BPE (es. "Ġhello") nella vera stringa Python (" hello")
            try:
                raw_bytes = bytes([self.byte_decoder[c] for c in bpe_token])
                decoded_str = raw_bytes.decode("utf-8", errors="replace")
            except KeyError:
                # Token speciali non-BPE
                decoded_str = bpe_token
            self.id_to_str[token_id] = decoded_str

    def encode_greedy(self, text: str) -> List[int]:
        """Codifica una stringa in token ID tramite Longest-Prefix Match sul vocabolario
(Bonus)."""
        bpe_repr = "".join(self.byte_encoder[b] for b in text.encode("utf-8"))
        token_ids: List[int] = []
        i = 0
        n = len(bpe_repr)
        while i < n:
            matched = False
            for j in range(n, i, -1):
                piece = bpe_repr[i:j]
                if piece in self.token_to_id:
                    token_ids.append(self.token_to_id[piece])
                    i = j
                    matched = True
                    break
            if not matched:
                i += 1  # carattere non presente nel vocabolario: lo si salta
        return token_ids

    def decode(self, token_ids: List[int]) -> str:
        """Ricostruisce la stringa reale concatenando le stringhe dei token."""
        return "".join(self.id_to_str.get(t, "") for t in token_ids)
```

4.3 Il Problema del Disallineamento dei Confini (Token Boundary Misalignment)

Come analizzato in dettaglio da **Willard & Louf (2023)** e da **Dong et al. (2024)** (nel paper di *XGrammar*), la sfida principale del Constrained Decoding nasce dal fatto che **i token BPE non coincidono con i terminali lessicali della grammatica JSON**:

- **Token multi-carattere che attraversano i confini**: Nel vocabolario esistono token come `'",'`, `'":'`, `'"}'`, `'12'` o `'true}'`. Se stiamo generando una stringa JSON e il modello emette il token `'hello"'`, quel singolo token contiene contemporaneamente il contenuto della stringa *e* le virgolette di chiusura che fanno avanzare lo stato dell'automa!
- **Token spezzati (Sub-token)**: Il nome della funzione `fn_add_numbers` non è un singolo token nel vocabolario di Qwen3, ma viene spezzato in più token, ad esempio `["fn", "_add", "_numbers"]`. Di conseguenza, durante la selezione della funzione non basta controllare il primo token: occorre mantenere un filtro di prefisso o calcolare lo score della sequenza completa.

5. Algoritmo di Constrained Decoding per Call Me Maybe

Vediamo ora come progettare un motore di Constrained Decoding su misura per `call me maybe` che sia al contempo **semplice da spiegare durante la valutazione**, **matematicamente infallibile** (100% JSON valido) e **velocissimo**.

5.1 Scomposizione Strutturale dello Schema (Divide et Impera)

Invece di scrivere un parser generico Context-Free Grammar (CFG) con stack di pushdown per qualsiasi JSON arbitrario (**Dong et al., 2024**), osserviamo che in `call me maybe` la struttura dell'oggetto da produrre per ciascun prompt è completamente nota a priori una volta letto `functions_definition.json`:

1. **Campo `prompt`**: È identico alla stringa di input! Non deve mai essere generato dall'LLM (generarlo con l'LLM rischierebbe di corromperlo o sprecare decine di token).
2. **Campo `name`**: Deve essere esattamente una delle stringhe f_1, f_2, …, f_k presenti in `functions_definition.json`.
3. **Campo `parameters`**: Una volta nota la funzione scelta f_*, l'elenco ordinato delle chiavi dei parametri (es. `["a", "b"]`) e il tipo di ciascun parametro (`"number"`, `"string"`, `"boolean"`) sono **100% noti dallo schema**!

Questo significa che l'intero processo di generazione guidata per un singolo prompt si articola in **due fasi vincolate sui logit** intercalate da **Jump-Forward deterministici**:

5.2 Fase 1: Selezione Vincolata del Nome della Funzione (Prefix Tree / Trie)

Forniamo all'LLM un prompt ben strutturato che descrive gli strumenti disponibili, la richiesta dell'utente e termina con l'inizio dell'oggetto JSON di chiamata:

```
{"name": "
```

A questo punto l'LLM deve generare il nome della funzione seguito da `"`. Poiché i nomi validi sono un insieme finito F = { `"fn_add_numbers"`, `"fn_greet"`, `"fn_reverse_string"`, …}, possiamo usare il Constrained Decoding basato su prefisso:

- Sia `generated_name = ""` la stringa accumulata finora per il nome della funzione.
- Ad ogni step, i candidati ancora compatibili sono F_{active} = { $f \in F$ | f inizia con `generated_name` }.
- Un token $v \in V$ è valido se e solo se la stringa `generated_name + token_str(v)` è un prefisso di almeno una stringa `f + '"'` con $f \in F_{active}$.
- **Ottimizzazione Deterministica (Trie Early-Commit)**: Non appena F_{active} si riduce a un'unica funzione (ad esempio dopo che il modello ha generato `fn_add`, l'unica funzione che inizia con `fn_add` è `fn_add_numbers`), non serve più interrogare l'LLM per gli altri sub-token del nome! Completiamo direttamente `fn_add_numbers"` con Jump-Forward.

5.3 Fase 2: Generazione Vincolata per Tipo dei Parametri (Type-Guided FSM)

Selezionata la funzione f_*, iteriamo sui suoi parametri definiti in `functions_definition.json`. Per il primo parametro (es. `"a"` di tipo `"number"`), appendiamo deterministicamente al contesto la stringa:

```
, "parameters": {"a":
```

e convertiamo il nuovo contesto in `input_ids`. Ora applichiamo il mascheramento dei logit in base al tipo `param_type` previsto dallo schema:

Caso A: Tipo `"number"` (Interi e Float JSON)

Secondo la specifica RFC 8259 (JSON), un numero valido segue l'espressione regolare `^-?(0|[1-9][0-9]*)(\.[0-9]+)?$`. Durante la decodifica vincolata di un numero:

- Pre-filtriamo all'avvio del programma il sottoinsieme del vocabolario `NUMERIC_CANDIDATE_TOKENS` contenente solo i token composti esclusivamente da cifre `0-9`, segno `-`, punto decimale `.` ed eventuali delimitatori di chiusura JSON (`,` o `}`). Su 151.936 token di Qwen3, i token puramente numerici/delimitatori sono meno di 1.500!
- Ad ogni passo, per ciascun token candidato v, simuliamo l'aggiunta di `token_str(v)` al buffer numerico corrente `num_buf`:
  - Se il token contiene il delimitatore di fine parametro (`,` se ci sono altri parametri dopo, oppure `}` se è l'ultimo parametro), verifichiamo che la parte precedente il delimitatore formi un numero completo valido (es. `"42"` o `"-3.14"`, ma non `""` né `"42."` né `"-"`). Se l'LLM sceglie questo token, la generazione del parametro termina.
  - Altrimenti, verifichiamo tramite un automa/regex di prefisso che il nuovo `num_buf` sia un prefisso valido di un numero JSON (es. un solo punto decimale, segno `-` ammesso solo in prima posizione, niente zeri multipli iniziali come `00`).
- Una volta chiuso il numero, convertiamo la stringa in `float` o `int` (nota: nell'esempio V.4.1 del subject, `{"a": 2.0, "b": 3.0}` usa `float` per il tipo `number`, oppure `int` / `float` nativi compatibili con lo

schema JSON).

Caso B: Tipo `"string"`

Per un parametro di tipo `string`, le virgolette di apertura `"` sono obbligatorie: le inseriamo subito nel Jump-Forward (`"s": "`). Durante la generazione del contenuto della stringa:

- Qualsiasi sequenza di caratteri UTF-8 stampabili è valida in JSON, purché i caratteri di controllo (come newline grezzi `\n` o tab `\t` non escapati) siano vietati e le sequenze di escape `\"` o `\\` siano ben formate.
- Quando un token contiene una virgoletta non preceduta da backslash `"`, solo i token in cui la virgoletta chiude esattamente il valore (o è seguita dal delimitatore atteso `,` o `}`) sono ammessi.
- *Guardrail contro stringhe vuote involontarie*: Se il prompt richiede di estrarre un argomento testuale non vuoto, al primo token del valore stringa possiamo opzionalmente impedire la chiusura immediata `""` se il prompt non menziona una stringa vuota, lasciando che il modello chiuda con `"` non appena completata la parola (es. `hello"`).

Caso C: Tipo `"boolean"`

Un booleano JSON ammette esclusivamente i due letterali `true` e `false`. Basta confrontare i logit dei token che iniziano con `true` e `false` in un singolo passo di decodifica!

5.4 Esempio Completo di Codice: Motore di Logit Masking Tipizzato

Di seguito è mostrato il nucleo algoritmico del decodificatore vincolato, conforme ai requisiti `flake8` e `mypy --strict` del progetto:

```
import re
from typing import Dict, List, Set, Tuple
import numpy as np
import numpy.typing as npt

# Regex che valida sia un numero JSON completo sia un prefisso parziale (es. "-" o "12.")
NUMBER_PREFIX_RE = re.compile(r"^-?(?:0|[1-9]\d*)?(?:\.\d*)?$")
NUMBER_COMPLETE_RE = re.compile(r"^-?(?:0|[1-9]\d*)(?:\.\d+)?$")


class ConstrainedGenerator:
    """Motore di Constrained Decoding basato su Logit Masking e Jump-Forward."""

    def __init__(self, id_to_str: Dict[int, str], vocab_size: int) -> None:
        self.id_to_str = id_to_str
        self.vocab_size = vocab_size
        # Pre-indicizzazione dei token contenenti caratteri numerici o chiusure JSON
        self.numeric_token_ids: List[int] = [
            tid for tid, s in id_to_str.items()
            if s and all(c in "0123456789-. ,}" for c in s)
        ]

    def select_function_name(
        self,
        model_logits_fn: object,
```

```
        input_ids: List[int],
        valid_names: List[str],
        encode_fn: object,
    ) -> str:
        """Seleziona il nome della funzione vincolando i logit al Trie dei nomi
validi."""
        # Calcola la maschera sui primi token di ciascun nome valido
        # Se un prefisso identifica univocamente una funzione, applica Jump-Forward
        generated = ""
        current_ids = list(input_ids)

        while True:
            candidates = [name for name in valid_names if name.startswith(generated)]
            if len(candidates) == 1:
                # Jump-Forward: rimane una sola funzione possibile nello schema!
                return candidates[0]

            # Ottieni i logit dal modello (1D list of floats)
            logits_list: List[float] = getattr(
                model_logits_fn, "get_logits_from_input_ids"
            )(current_ids)
            logits: npt.NDArray[np.float64] = np.array(logits_list, dtype=np.float64)

            mask: npt.NDArray[np.bool_] = np.zeros(len(logits), dtype=bool)
            for tid, token_str in self.id_to_str.items():
                if not token_str or tid >= len(logits):
                    continue
                # Valido se (generated + token_str) è prefisso di almeno un nome + '"'
                if any((name + '"').startswith(generated + token_str) for name in
candidates):
                    mask[tid] = True

            # Greedy: argmax sui soli token validi (nessun Softmax necessario)
            masked_logits = np.where(mask, logits, -np.inf)
            best = int(np.argmax(masked_logits))
            generated += self.id_to_str[best]
            current_ids.append(best)

            if generated.endswith('"'):
                return generated[:-1]
```

6. Prompt Engineering Strutturato per Small Language Models (SLMs)

Un errore concettuale frequente consiste nel pensare che il Constrained Decoding renda superfluo il design del prompt. Come dimostrato empiricamente da **Geng et al. (2023)** e **Tam et al. (2024)**, il Constrained Decoding garantisce il **100% di validità sintattica**, ma è la distribuzione di probabilità condizionata dal prompt a determinare l'**accuratezza semantica** (ovvero se il modello sceglierà `fn_add_numbers` invece di `fn_reverse_string` e se estrarrà `265` invece di un numero inventato).

6.1 Allineamento con il Pre-training di Qwen3-0.6B

`Qwen/Qwen3-0.6B` è stato addestrato con il template conversazionale **ChatML** e possiede capacità native di comprensione degli strumenti (*tool calling*) (**Yang et al., 2025**). Tuttavia, Qwen3 introduce anche una modalità di ragionamento (`<think>...</think>`) che, se lasciata libera, consumerebbe centinaia di token prima di arrivare alla risposta. Nel nostro caso, poiché forziamo immediatamente l'inizio del JSON alla fine del prompt, disattiviamo implicitamente qualsiasi prolissità conversazionale.

Per massimizzare la precisione dei logit di `Qwen3-0.6B` nell'estrazione degli argomenti, il prompt di input passato a `encode()` dovrebbe seguire una struttura chiara, compatta e con un singolo esempio *one-shot* astratto (non hardcodato sui test specifici):

```
Available functions:
- fn_add_numbers(a: number, b: number): Add two numbers together and return their sum.
- fn_greet(name: string): Generate a greeting message for a person by name.
- fn_reverse_string(s: string): Reverse a string and return the reversed result.

Task: Select the correct function and extract the exact parameter values from the user
prompt as JSON.
User prompt: "What is the sum of 265 and 345?"
JSON call: {"name": "
```

Perché questo formato funziona così bene con un modello da 0.6B parametri?

- **Riduce la distanza di attenzione (Attention Span)**: Invece di incollare l'intero `functions_definition.json` grezzo (pieno di parentesi graffe annidate e `"returns": {"type": ...}` che confondono le teste di attenzione di un modello piccolo), formattiamo ogni funzione su una singola riga leggibile `nome(param: tipo): descrizione`.
- **Posiziona il Prompt Utente subito prima del JSON**: Nei modelli da 0.6B, i layer di induzione (*Induction Heads*) copiano i token letterali (come i numeri `265` o la stringa `'hello'`) molto più accuratamente quando il testo sorgente si trova a pochi token di distanza dal punto di generazione.

7. Validazione con Pydantic v2, Gestione Errori e Ottimizzazione

Il capitolo IV.3.1 del subject stabilisce un requisito tassativo: **"All classes must use pydantic for validation"** e **"Your program must never crash unexpectedly and must always provide clear error messages to the user"**.

7.1 Modellazione dello Schema con Pydantic v2

Invece di usare classi Python standard o `dataclass`, ogni struttura dati del progetto (configurazione CLI, definizioni delle funzioni di input, prompt di test, risultato dell'invocazione e stato del decodificatore) deve ereditare da `pydantic.BaseModel`:

```
from typing import Any, Dict, List, Literal
from pydantic import BaseModel, ConfigDict, Field, ValidationError


class ParameterSpec(BaseModel):
```

```
    """Specifica di un singolo parametro di funzione in functions_definition.json."""
    model_config = ConfigDict(extra="forbid")
    type: Literal["number", "integer", "string", "boolean", "array", "object"]


class ReturnSpec(BaseModel):
    """Specifica del valore di ritorno di una funzione."""
    model_config = ConfigDict(extra="allow")
    type: str


class FunctionDefinition(BaseModel):
    """Modello Pydantic per validare ciascuna funzione disponibile."""
    model_config = ConfigDict(extra="forbid")
    name: str = Field(min_length=1)
    description: str
    parameters: Dict[str, ParameterSpec]
    returns: ReturnSpec


class PromptTestItem(BaseModel):
    """Singolo elemento del file function_calling_tests.json."""
    model_config = ConfigDict(extra="forbid")
    prompt: str


class FunctionCallOutput(BaseModel):
    """Formato rigoroso richiesto per ciascun elemento dell'output finale (Sez. V.4)."""
    model_config = ConfigDict(extra="forbid")
    prompt: str
    name: str
    parameters: Dict[str, Any]
```

7.2 Architettura delle Ottimizzazioni per la Moulinette (< 5 Minuti)

Durante la peer review, il programma viene testato su hardware standard (spesso privo di GPU dedicata, eseguendo l'inferenza su CPU). Per garantire tempi di risposta nell'ordine dei secondi:

1. **Singleton dell'istanza `Small_LLM_Model` e Pre-processing del Vocabolario**: Caricare il modello e parsare i 151.936 token di `vocab.json` richiede circa 2–4 secondi. Questa operazione va eseguita **una sola volta** all'avvio del programma, mai dentro il ciclo dei prompt.
2. **Pre-calcolo delle Maschere Statiche**: Le maschere booleane per i token numerici, per i booleani e per i token di contenuto stringa (privi di virgolette non escapate e caratteri di controllo) non dipendono dal prompt! Calcolarle una sola volta nel costruttore come array NumPy booleani permette di valutare la validità in $O(k)$ su poche centinaia di candidati anziché iterare su 151.936 elementi a ogni token generato (**Willard & Louf, 2023**; **Dong et al., 2024**).
3. **Jump-Forward su tutti i Delimitatori JSON**: Come illustrato nella Sezione 2.2 (**Zheng et al., 2023**), non usare mai l'LLM per generare `{"prompt": ...}`, né per generare i nomi dei parametri `"a":` o `"b":`. L'LLM viene interrogato esclusivamente per scegliere il nome della funzione (spesso 1 singolo step grazie al Trie Early-Commit!) e per generare i token del valore di ciascun argomento (1–3 step per parametro). In

media, un intero oggetto JSON viene completato con **sole 3–5 chiamate totali** a `get_logits_from_input_ids()`!

8. Riferimenti Bibliografici (Letteratura Scientifica e arXiv)

1. **Willard, B. T., & Louf, R. (2023).** *Efficient Guided Generation for Large Language Models.* arXiv preprint arXiv:2307.09702. Disponibile su: https://arxiv.org/abs/2307.09702 (Paper fondamentale che introduce l'indicizzazione del vocabolario tramite Automi a Stati Finiti per il Constrained Decoding a costo zero, alla base della libreria *Outlines*).
2. **Geng, S., Josifoski, M., Peyrard, M., & West, R. (2023).** *Grammar-Constrained Decoding for Structured NLP Tasks without Finetuning.* In Proceedings of EMNLP 2023, arXiv:2305.13971. Disponibile su: https://arxiv.org/abs/2305.13971 (Dimostra come i modelli pre-addestrati di piccole dimensioni raggiungano affidabilità strutturale del 100% senza fine-tuning grazie ai vincoli grammaticali sui logit).
3. **Zheng, L., Yin, L., Xie, Z., Sun, C., Huang, J., Yu, C. H., Cao, S., Kozyrakis, C., Stoica, I., Gonzalez, J. E., Barrett, C., & Sheng, Y. (2023).** *SGLang: Efficient Execution of Structured Language Model Programs.* arXiv preprint arXiv:2312.07104. Disponibile su: https://arxiv.org/html/2312.07104 (Introduce la tecnica di *Compressed Finite State Machine* e *Jump-Forward Decoding* per saltare le forward pass sui segmenti deterministici di uno schema JSON).
4. **Dong, Y., Ruan, C. F., Cai, Y., Lai, R., Xu, Z., Zhao, Y., & Chen, T. (2024).** *XGrammar: Flexible and Efficient Structured Generation Engine for Large Language Models.* arXiv preprint arXiv:2411.15100. Disponibile su: https://arxiv.org/pdf/2411.15100 (Analizza la separazione tra token indipendenti dal contesto e dipendenti dal contesto e risolve il problema del disallineamento dei confini dei token BPE).
5. **Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017).** *Attention Is All You Need.* Advances in Neural Information Processing Systems (NeurIPS), 30, arXiv:1706.03762.
6. **Sennrich, R., Haddow, B., & Birch, A. (2016).** *Neural Machine Translation of Rare Words with Subword Units.* In Proceedings of ACL 2016, arXiv:1508.07909 (Introduce l'algoritmo Byte-Pair Encoding per la tokenizzazione NLP).
7. **Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., & Sutskever, I. (2019).** *Language Models are Unsupervised Multitask Learners.* OpenAI Technical Report (Introduce il Byte-Level BPE con mappatura byte-to-unicode usato anche da Qwen).
8. **Yang, A., et al. — Qwen Team (2025).** *Qwen3 Technical Report.* Alibaba Group / arXiv preprint (Descrive l'architettura della famiglia Qwen3, incluso il modello denso `Qwen3-0.6B` con vocabolario BBPE da 151.936 token e supporto nativo al tool calling).
9. **Tam, Z. R., Wu, C.-K., Tsai, Y.-L., Lin, C.-Y., Lee, H.-Y., & Chen, Y.-N. (2024).** *Let Me Speak Freely? A Study on the Impact of Format Restrictions on Performance of Large Language Models.* arXiv preprint arXiv:2408.02442 (Analizza l'interazione tra il formato del prompt e i vincoli di decodifica sull'accuratezza del ragionamento).